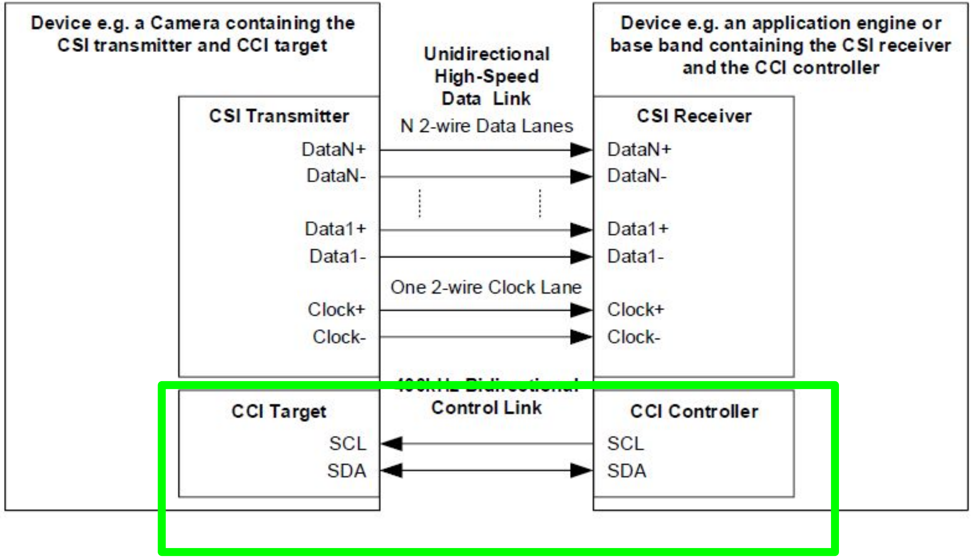
Design and Implementation of CSI Controller IP

SI-VISION ACADEMY

Camera Control Interface

CCI-Camera Control Interface

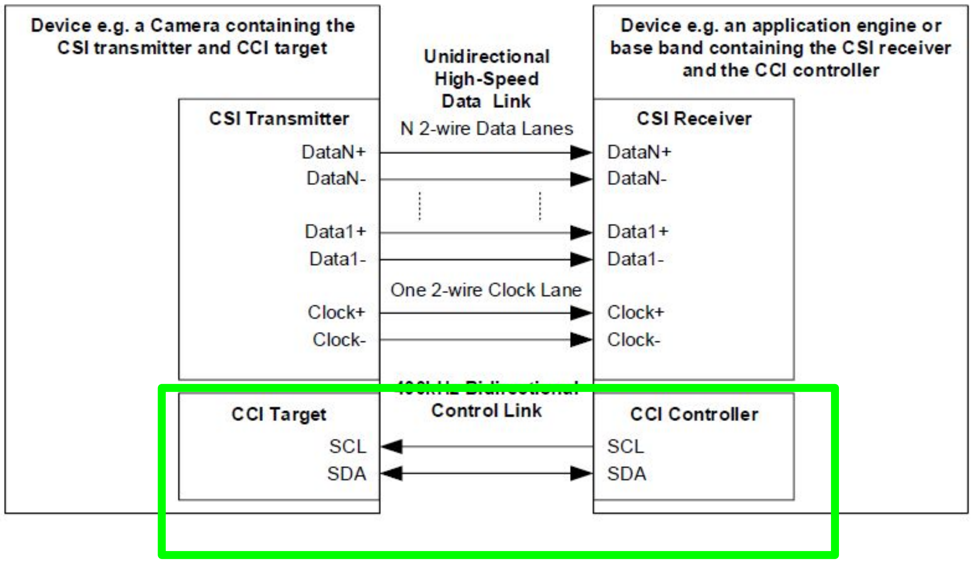
- The Camera Control Interface (CCI) is an I²C-based control bus used by the host processor to configure and control MIPI-compatible image sensors.
- It provides access to internal sensor registers (CCI Registers) that define operational parameters such as exposure, gain, and mode control.



CCI-Camera Control Interface

Case 1 :CSI transmitter IS inside the sensor (normal case)

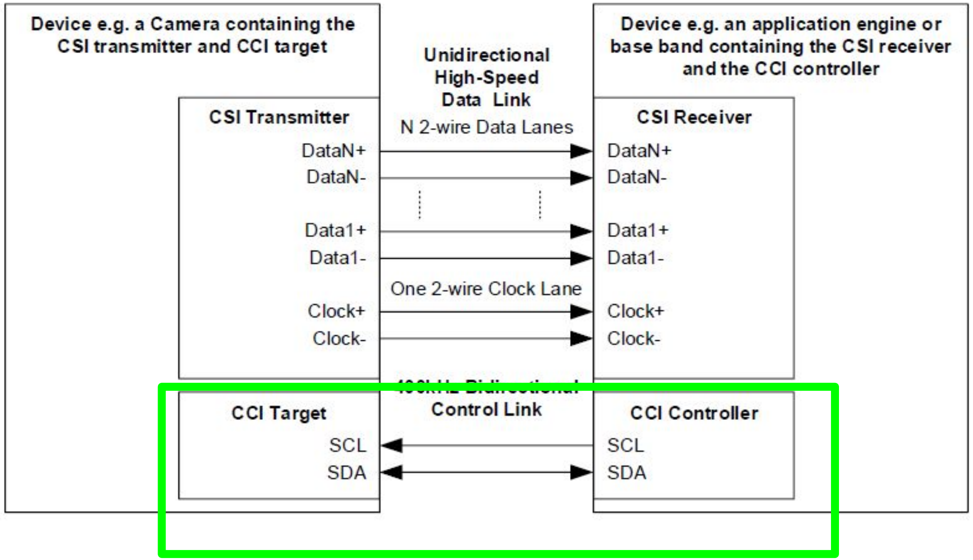
- Sensor → CSI TX → D-PHY → SoC
- CCI (I^2C) configures the sensor, including its CSI TX.



CCI-Camera Control Interface

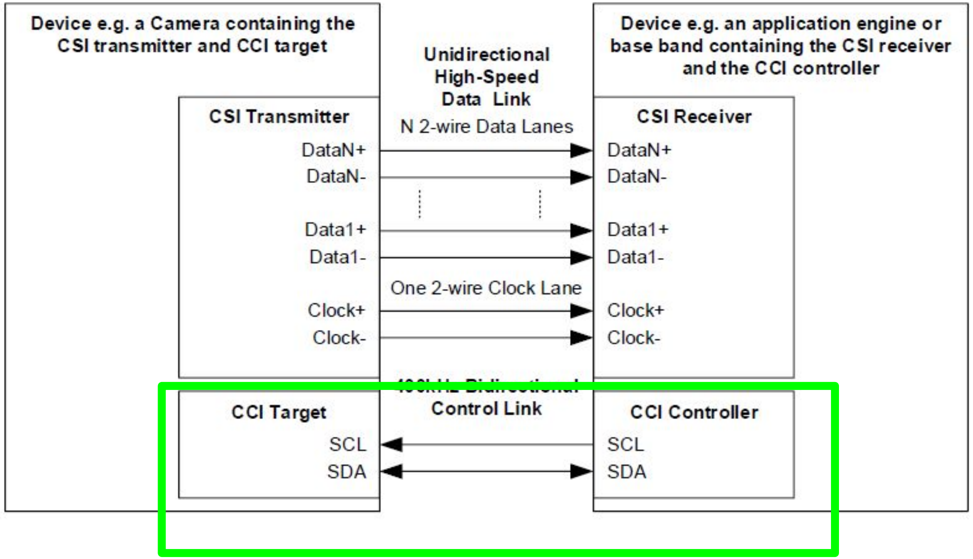
Case 2:CSI transmitter is NOT inside the sensor

- The camera sensor outputs parallel data (RAW/RGB)
- An external chip converts it to CSI-2
- Sensor → RAW → [External CSI-2 Transmitter] → D-PHY→ SoC
- The sensor has NO CSI registers (Because it doesn't generate CSI).
  It only has registers related to:
  exposure/gain
  frame size
  timing signals (VSYNC/HSYNC/pclk)
  pixel format (RAW/Bayer/YUV)
- The external CSI transmitter has its OWN register file



CCI-Camera Control Interface

- The CCI does NOT control CSI.
- CCI does NOT write into any CSI-2 register file.
- CCI only writes into the sensor's internal registers,These registers control how the sensor generates CSI-2 data, but they do not configure the CSI receiver or CSI transmitter logic itself.

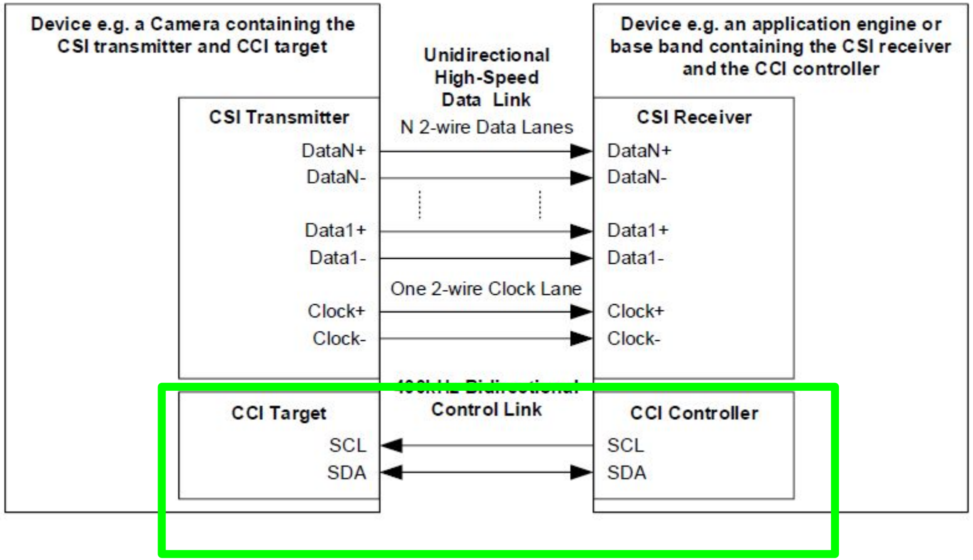


CCI-Camera Control Interface

Option1: SoC configures it using its own I²C/SPI bus

Option2: CSI transmitter is connected to the same I²C bus as the sensor

- sensor has its own I²C address
- CSI transmitter has its own address
- Both devices are slaves on the same bus, but configured independently.



CSI_configuration Registers-TX

Top-Level Architecture Overview

TX Architecture requires a Register Bank (APB/AHB Slave) to allow software to configure the transmission. These registers drive three main internal blocks:

1-Protocol Layer: Handles Header generation (VC, DT) and CRC.

2-Lane Management: Distributes bytes across 1, 2, or 4 lanes.

3-D-PHY Controller: Manages the Analog Front End (AFE) timings (HS-Prepare, HS-Settle).

CSI_configuration Registers-TX

Group A: Global Control & Reset
Purpose: These registers control the Finite State Machine (FSM) start/stop sequences.

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_COMMON_CONTROL**

TX Implementation: **TX_GLOBAL_CTRL**

Bit [0] ENABLE: Master switch. When set high, the TX state machine moves from IDLE to operational.

Bit [1] SOFT_RESET: Clears internal FIFOs and resets the Protocol FSM.

Bit [2] LANE_COUNT: (2 bits). Configures the lane distributor.
00: 1 Lane
01: 2 Lanes
11: 4 Lanes

Bit [4] CONT_CLK_MODE: (Continuous Clock).
Design Impact: If High, the Clock Lane remains in High Speed (HS) mode even during data gaps (LP-11). If Low, the Clock Lane enters Low Power mode between frames.

CSI_configuration Registers-TX

Group B: Protocol Configuration (The Packetizer)
Purpose: These registers determine what data is written into the Packet Header (PH).

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_ISP_CONFIG_CHn** (Channel Config)

TX Implementation: **TX_PROTOCOL_CFG**

Bits [5:0] DATA_TYPE (DT): The 6-bit ID indicating the format (e.g., 0x2B for RAW10, 0x24 for RGB888).

Interaction: The Packetizer hardware reads this register and inserts it inPacket Header.

Bits [7:6] VIRTUAL_CHANNEL (VC): The 2-bit ID (0-3).

Bits WORD_COUNT (WC):

CSI_configuration Registers-TX

Group B: Protocol Configuration (The Packetizer)
Purpose: These registers determine what data is written into the Packet Header (PH).

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_ISP_CONFIG_CHn** (Channel Config)

TX Implementation: **TX_PROTOCOL_CFG**

Bits [5:0] DATA_TYPE (DT): The 6-bit ID indicating the format (e.g., 0x2B for RAW10, 0x24 for RGB888).

Interaction: The Packetizer hardware reads this register and inserts it in Packet Header.

Bits [7:6] VIRTUAL_CHANNEL (VC): The 2-bit ID (0-3).

Bits WORD_COUNT (WC):

CSI_configuration Registers-TX

Group C: D-PHY Timing Configuration
Purpose: This is the most critical section for electrical compliance. The RX expects specific setup times (`T_HS-SETTLE`). The TX must generate specific drive times (`T_HS-PREPARE` + `T_HS-ZERO`).
Ref (i.MX 8MP): **`MIPI_CSI2_CSIS_DPHY_COMMON_CONTROL`**

TX Implementation:**`TX_DPHY_TIMING`**

HS_PREPARE: Sets the time the TX drives LP-00 before starting the HS burst.

HS_ZERO: Sets the duration of the sequence of zeros before the Sync Byte (0xB8).

HS_TRAIL: Sets the flip time after the packet ends.

Status & Debugging -TX

The i.MX RX has extensive error reporting (`INT_MSK`, `INT_SRC`). The TX needs the counterpart registers to *report* issues to the host processor.

TX Implementation: `TX_INT_STATUS`

- **FIFO_UNDERRUN:** Critical. Indicates the pixel interface (from sensor) is too slow, and the CSI TX ran out of data mid-packet.
- **FIFO_OVERFLOW:** The CSI TX cannot push data out to the D-PHY fast enough (Lane frequency too low).
- **FSM_STATE:** A read-only field showing the FSM state

CSI_configuration Registers-RX

Top-Level Architecture Overview

RX Register Bank needs to control three stages of "Data Recovery":

Stage 1: The Analog Front End (D-PHY): Tuning the electrical reception (Termination & Settle Time).

Stage 2: The Lane Merger: Re-assembling bytes from 1, 2, or 4 lanes into a single stream.

Stage 3: The Protocol Filter: Deciding which packets to keep (based on VC/DT) and which to drop.

CSI_configuration Registers-RX

Group A: PHY Calibration (The "Blindfold")
Focus: Signal Integrity.
Key Reg: `RX_DPHY_TIMING` (Controls `T_HS-SETTLE`).
Function: Ignores the noisy transition from Low Power to High Speed.

Group B:Global Configuration
Focus: Link Topology.
Key Reg: `RX_GLOBAL_CTRL`.
Function*:* Sets Number of Lanes (must match TX) and resets the "Byte Alignment" logic.

Group C: Packet Filtering
Focus: Data Logic.
Key Reg: RX_PROTOCOL_CFG.
Function: Defines which Virtual Channel (VC) and Data Type (DT) are valid.
Action: If incoming VC != Register VC, the hardware drops the packet automatically.

CSI_configuration Registers-RX

Essential RX Register Definitions

RX_DPHY_TIMING

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_DPHY_COMMON_CONTROL**

Bit [7:0] HS_SETTLE_COUNT:

Definition: The number of nanoseconds (or clock cycles) the RX ignores the D-PHY lines after detecting the start of a High-Speed burst.

Usage: Software must tune this to be shorter than the TX's T_HS-PREPARE + T_HS-ZERO.

Failure Mode: If set too high, RX misses the Sync Byte (Start of Packet).

CSI_configuration Registers-RX

Essential RX Register Definitions

RX_GLOBAL_CTRL

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_COMMON_CONTROL**

Bit [0] ENABLE: Enable.

Bit [1] SOFT_RESET: Resets the CRC/ECC Error Counters and FIFO pointers.

Bit [3:2] LANE_COUNT:

00: 1 Lane | 01: 2 Lanes | 11: 4 Lanes.

Hardware Logic: This controls the "lane merger" that merges parallel lane data.).

CSI_configuration Registers-RX

Essential RX Register Definitions

.**RX_PROTOCOL_CFG** (Double Buffered)

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_ISP_CONFIG_CHn**

Bit [5:0] DATA_TYPE_MASK:

Bit [7:6] VC_ID: The specific Virtual Channel (0-3) this RX logic should capture.

Note:a "Shadow Register." Software writes to the shadow, and the hardware updates the active config only at the Frame End to prevent corruption mid-image.

CSI_configuration Registers-RX

Essential RX Register Definitions

.**RX_ERROR_STATUS** (Debug)

Ref (i.MX 8MP): **MIPI_CSI2_CSIS_INT_SRC**

ECC_SINGLE_ERR: Corrected 1-bit error in Header.

ECC_MULTI_ERR: Uncorrectable 2-bit error (Drop Packet).

CRC_ERR: Payload data corruption.

SOT_ERR: Start of Transmission Error (Synchronization failure)

Initialization

Initialization Phase:

- Software writes to **`TX_DPHY_TIMING`**.
- **Effect:** The D-PHY Analog block calibrates the lines.

Configuration Phase:

- Software writes `WC` (Word Count), `DT` (Data Type), and `VC` to **`TX_PROTOCOL_CFG`**.
- **Effect:** The **Packet Builder Block** latches these values.

Transmission Start:

- Software sets `ENABLE` in **`TX_GLOBAL_CTRL`**.
- **Effect:**
    1. The FSM wakes up.
    2. The Packet Builder creates a Short Packet (FS).
    3. The Lane Distributor takes the packet, splits bytes (if Multi-Lane), and sends them to the D-PHY.
    4. The D-PHY executes the `LP-11` -> `LP-01` -> `LP-00` (HS-Request) sequence based on timings in `TX_DPHY_TIMING`.

Initialization

1.System Power-Up (T=0)

- **Action:** Release Reset for both IP blocks.
- **Critical State:**
  - **TX D-PHY:** Must immediately drive **LP-11 (Stop State)**.
  - **RX D-PHY:** Must be in **LP-RX Mode** (listening for LP-11).

2. RX Phase

- **Action:** Configure RX Registers (Lanes, Format).
- **Command:** Set `RX_ENABLE = 1`.
- **Status Check:** RX FSM enters **`WAIT_FOR_STOP_STATE`**.
- *Architecture Note:* The RX **must** be fully active and waiting *before* the TX is enabled. If TX starts first, the RX will miss the "SoT" (Start of Transmission) pattern and discard the entire first frame.

Initialization

3. TX Phase

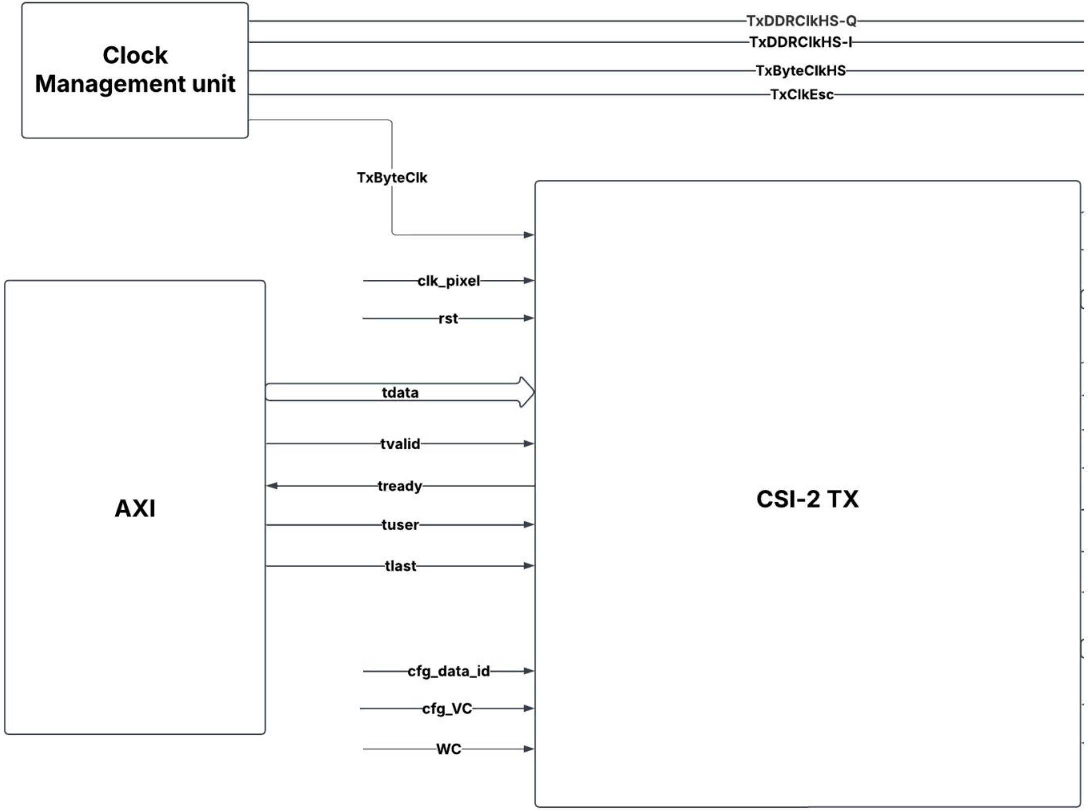
- **Action:** Configure TX Registers (matching RX settings).
- **Command:** Set `TX_ENABLE = 1`.
- **Step A (Clock):** TX Clock Lane goes `LP-11` → `HS-Request` → `HS-Clock`.
- **Step B (Lock):** RX D-PHY detects Clock, locks PLL. `RX_PHY_LOCK` bit goes high.

4. Data Transmission

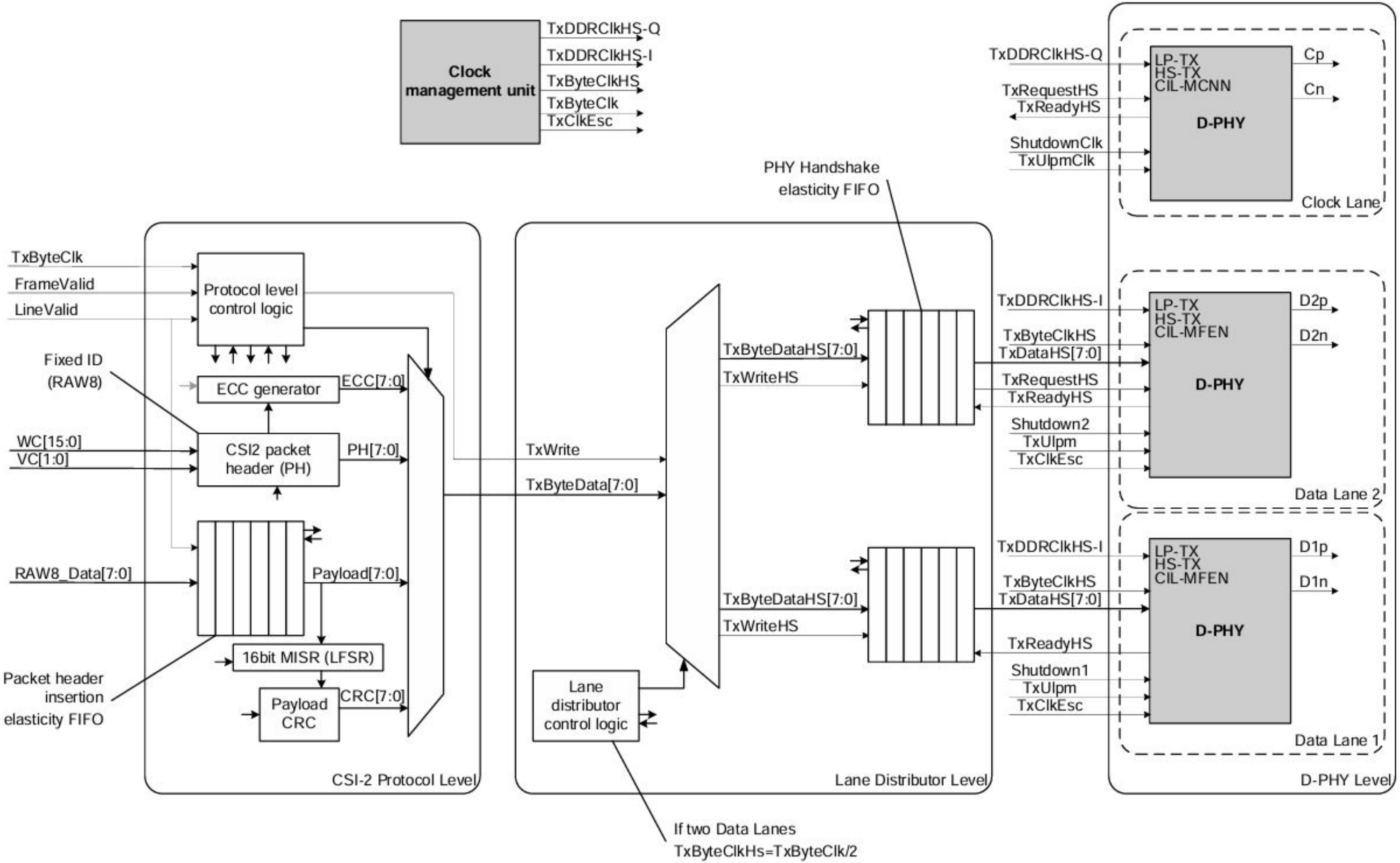
- **Trigger:** Once `RX_PHY_LOCK` is valid , TX starts Data Lane `HS-Burst`.
- **Result:** Clean capture of Frame 0.

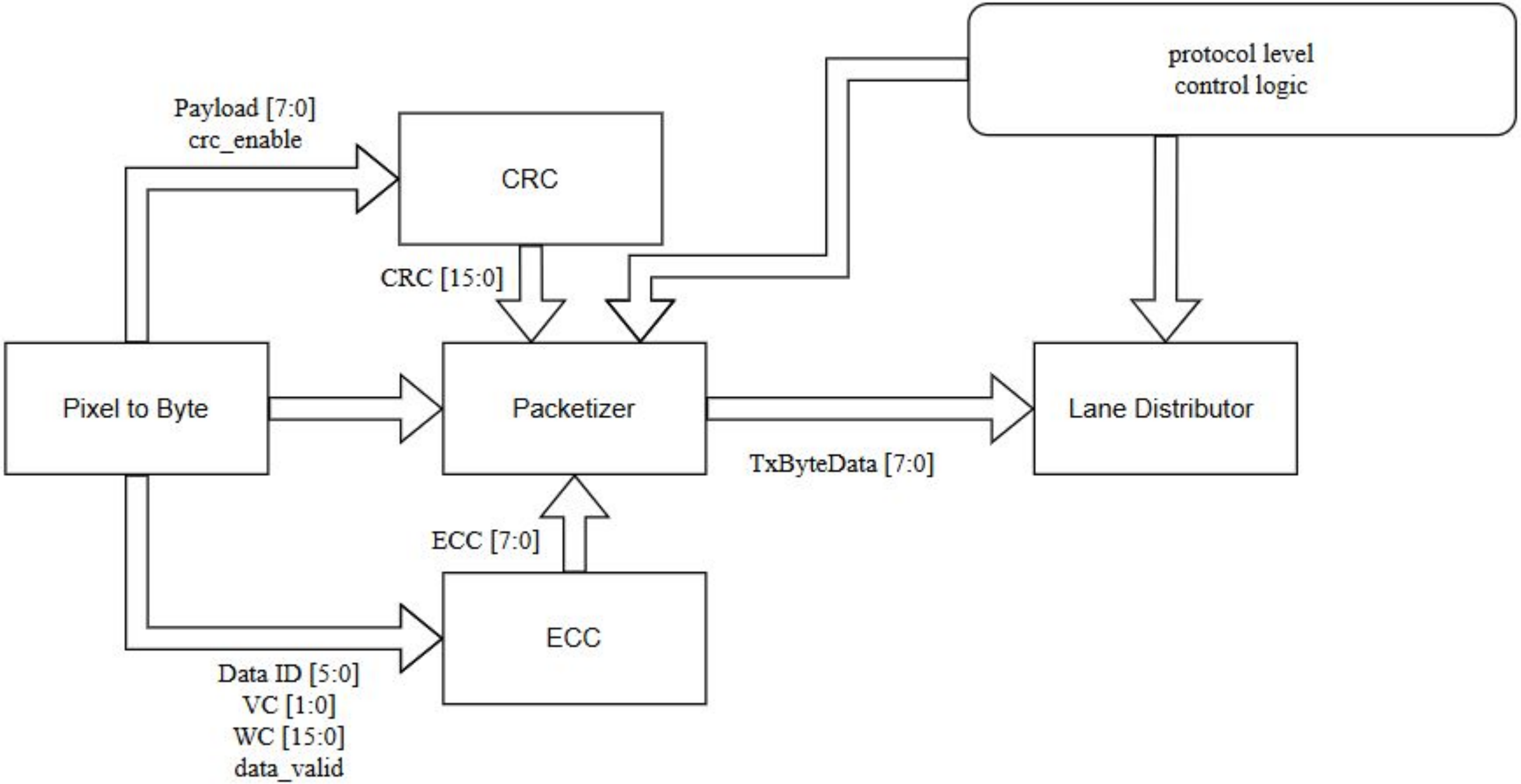
AXI4 Stream

CSI Transmitter interface with AXI



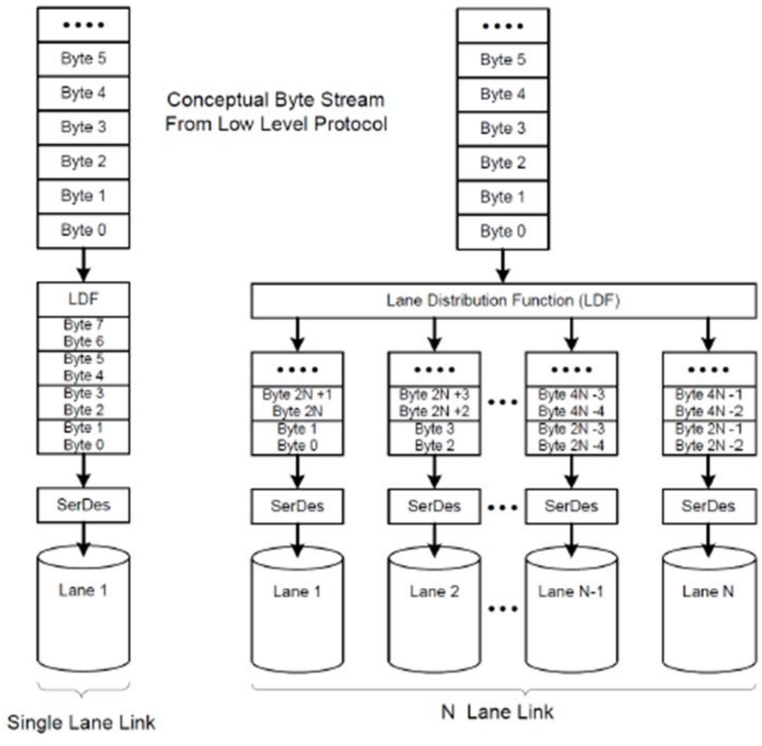
Flow

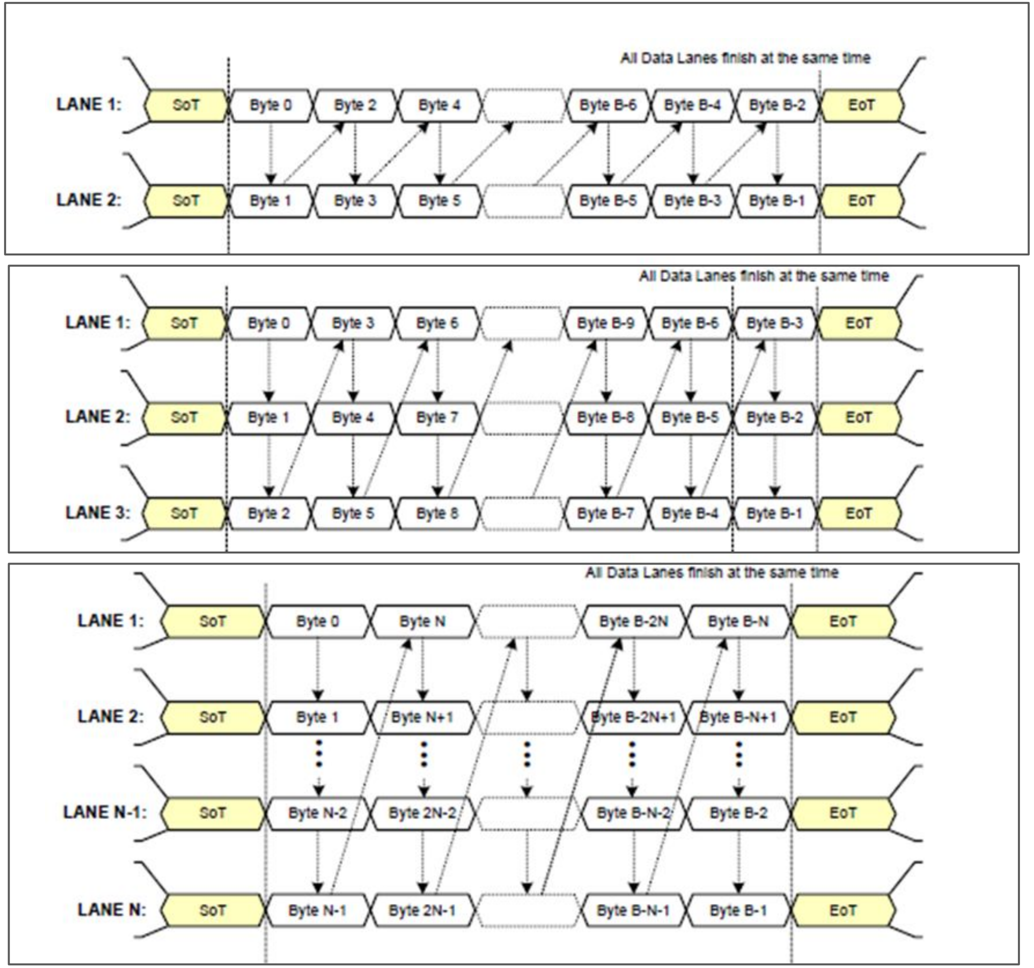

Clock management unit
TxDDRClkHS-Q
TxDDRClkHS-I
TxByteClkHS
TxByteClk
TxClkEsc
TxByteClk
FrameValid
LineValid
Protocol level control logic
Fixed ID (RAW8)
ECC generator
ECC[7:0]
WC[15:0]
VC[1:0]
CSI2 packet header (PH)
PH[7:0]
RAW8_Data[7:0]
Payload[7:0]
16bit MISR (LFSR)
Payload CRC
CRC[7:0]
Packet header insertion elasticity FIFO
CSI-2 Protocol Level
TxWrite
TxByteData[7:0]
PHY Handshake elasticity FIFO
TxByteDataHS[7:0]
TxWriteHS
Lane distributor control logic
Lane Distributor Level
If two Data Lanes TxByteClkHs=TxByteClk/2
TxDDRClkHS-Q
TxRequestHS
TxReadyHS
ShutdownClk
TxUlpmClk
LP-TX HS-TX CIL-MCNN
D-PHY
Cp
Cn
Clock Lane
TxDDRClkHS-I
TxByteClkHS
TxDataHS[7:0]
TxRequestHS
TxReadyHS
Shutdown2
TxUlpm
TxClkEsc
LP-TX HS-TX CIL-MFEN
D-PHY
D2p
D2n
Data Lane 2
TxDDRClkHS-I
TxByteClkHS
TxDataHS[7:0]
TxReadyHS
Shutdown1
TxUlpm
TxClkEsc
LP-TX HS-TX CIL-MFEN
D-PHY
D1p
D1n
Data Lane 1
D-PHY Level

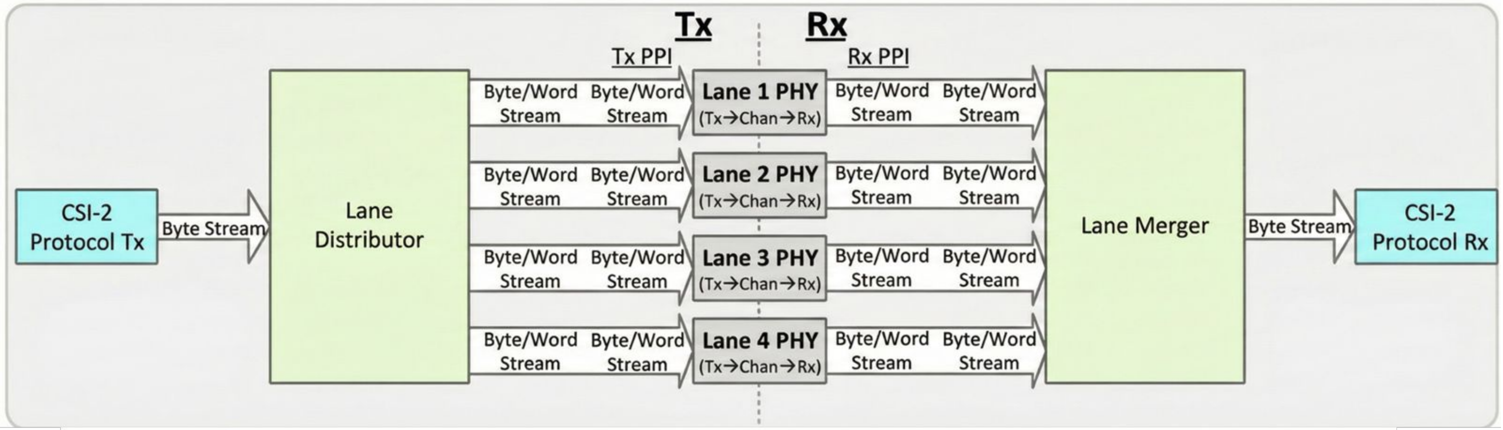

protocol level
control logic
Payload [7:0]
crc_enable
CRC
CRC [15:0]
Pixel to Byte
Packetizer
Lane Distributor
TxByteData [7:0]
ECC [7:0]
ECC
Data ID [5:0]
VC [1:0]
WC [15:0]
data_valid

Lane Distribution

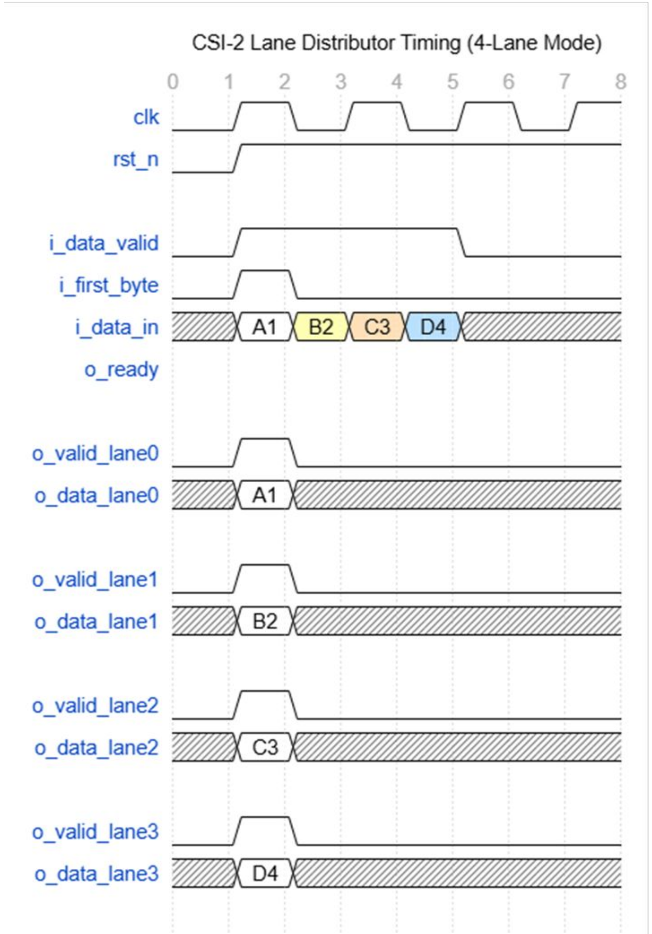
Lane Distributor





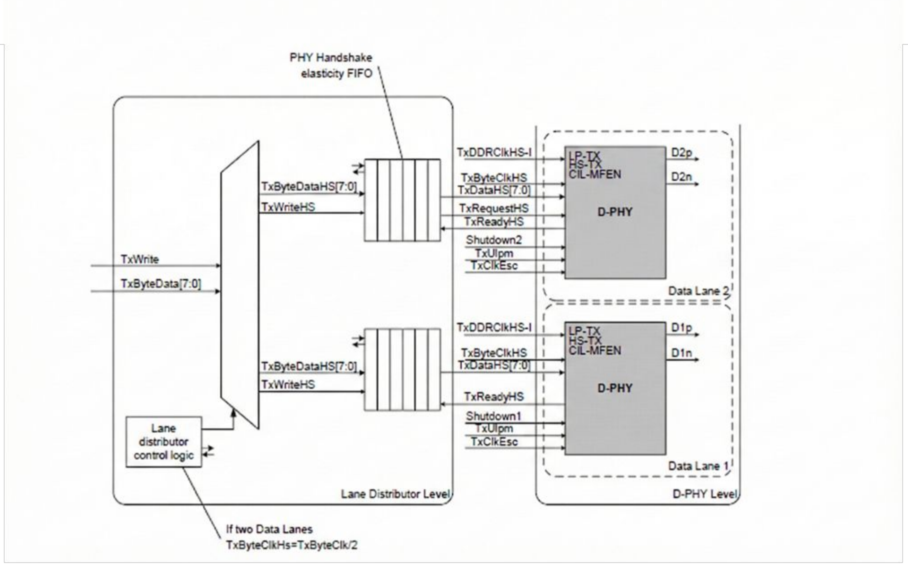

Tx
Rx
Tx PPI
Rx PPI
CSI-2 Protocol Tx
Byte Stream
Lane Distributor
Byte/Word Stream
Lane 1 PHY (Tx→Chan→Rx)
Lane 2 PHY (Tx→Chan→Rx)
Lane 3 PHY (Tx→Chan→Rx)
Lane 4 PHY (Tx→Chan→Rx)
Lane Merger
CSI-2 Protocol Rx


clk
rst_n
Data_In [7:0]
Data_Valid_In
First_Byte
Lane_Mode [1:0]
Ready_Out
Lane Distributor
Data_Out_Lane0 [7:0]
Valid_Out_Lane0
Data_Out_Lane1 [7:0]
Valid_Out_Lane1
Data_Out_Lane2 [7:0]
Valid_Out_Lane2
Data_Out_Lane3 [7:0]
Valid_Out_Lane3



CSI-2 Lane Distributor Timing (4-Lane Mode)
0 1 2 3 4 5 6 7 8
clk
rst_n
i_data_valid
i_first_byte
i_data_in
A1 B2 C3 D4
o_ready
o_valid_lane0
o_data_lane0
A1
o_valid_lane1
o_data_lane1
B2
o_valid_lane2
o_data_lane2
C3
o_valid_lane3
o_data_lane3
D4

clk
rst_n
Data_In [7:0]
Data_Valid_In
First_Byte
Lane_Mode [1:0]
Ready_Out
Lane Distributor
Data_Out_Lane0 [7:0]
Valid_Out_Lane0
Data_Out_Lane1 [7:0]
Valid_Out_Lane1
Data_Out_Lane2 [7:0]
Valid_Out_Lane2
Data_Out_Lane3 [7:0]
Valid_Out_Lane3



PHY Handshake elasticity FIFO
TxWrite
TxByteData[7:0]
TxByteDataHS[7:0]
TxWriteHS
Lane distributor control logic
Lane Distributor Level
If two Data Lanes TxByteClkHs=TxByteClk/2
TxDDRClkHS-I
TxByteClkHS
TxDataHS[7:0]
TxRequestHS
TxReadyHS
Shutdown2
TxUlpm
TxClkEsc
Shutdown1
LP-TX
HS-TX
CIL-MFEN
D-PHY
D2p
D2n
D1p
D1n
Data Lane 2
Data Lane 1
D-PHY Level

Error Handling

Error Handling

Corrected ECC error

When this error is detected, the CECCERR interrupt is triggered if CECCERRIE = 1 in CSI_IER0. This is an information event rather than a real error, as the CSI-2 Host controller successfully recovered the ECC error, so the frame is processed properly. Some details of the error are located in CECCVCERR and CECCDTERR in CSI_ERR1:

ECC error

When a multi-bit ECC error is detected, the ECCERR interrupt is triggered if ECCERRIE = 1 in CSI_IER0. As all virtual channels are potentially corrupted, the LPP immediately stops the current frames.

The CSI-2 Host controller restarts each enabled virtual channel on the next SOF packet of each channel. The software must perform the necessary error management (such as resetting some peripherals after this error detection). No details are available for the error as the relevant information is corrupted.

Error Handling

CRC error

When a CRC error is detected, the CRCERR interrupt is triggered if CRCERRIE = 1 in CSI_IER0. As the frame envelope is not corrupted, the CSI-2 Host controller does not stop the frame processing. The software must perform the necessary error management (such as reseting some peripherals, possibly including the CSI-2 Host controller). Details are available for this error in CRCVCERR and CRCDTERR in CSI_ERR1.

Data ID information

When a reserved data type is detected, the IDERR interrupt is triggered if IDERRIE = 1 in CSI_IER0. This is an information event rather than a real error, as the hardware processes this packet as for any non-erroneous packet.Details for the error are available in IDVCERR and IDDTERR in CSI_ERR1.

Error Handling

Shorter than expected packet

When a shorter-than-expected packet is received, the SPKTERR interrupt is triggered if SPKTERRIE = 1 in CSI_IER0. The packet is padded up to its theoretical size, and the frame is processed as if no error occurred. The software must perform the necessary error management (such as reseting some peripherals, possibly including the CSI decoder). Details for the error are available in SPKTVCERR and SPKTDTERR in CSI_ERR2.

References

References

[1] MIPI Alliance, "Specification for Camera Serial Interface 2 (CSI-2®) Version 4.0.1," MIPI Alliance, May 23, 2022

[2] MIPI Alliance, "Specification for D-PHY Version 2.5," MIPI Alliance, Jul. 5, 2019. [Online].

[3] Gowin MIPI Byte to Pixel Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2753/document/6820dee2cd163.pdf

[4] Intel Corporation, *"MIPI CSI-2 Intel® FPGA IP User Guide,"* Doc. No. 813926, Version 1.1.0, Apr. 26, 2024. Online. Available: https://www.intel.com/content/www/us/en/docs/programmable/813926.html

[5] Gowin MIPI Pixel to Byte Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2752/document/6820df0ae4095.pdf

Thank You!